SpiralTrain
developer courses

Course Schedule

- **Advanced Classes**

- Python Object Orientation
- Object Oriented Concepts
- Creating Classes
- Class Members
- Creating and Using Objects
- Property Syntax
- Built-In Class Attributes
- Destroying Objects
- Static Methods
- Encapsulation
- Data Hiding
- Inheritance
- Deriving Classes
- Constructor Chaining
- Multiple Inheritance
- Overriding Methods
- Operator Overloading
- Polymorphism
- Abstract Classes

Python Object Orientation

- **Python is an object-oriented language :**
  - Creating and using classes and objects is built in the language
  - Classes are created in `Python` with the `class` keyword
- **Class :**
  - Blueprint for a set of related objects with both description and implementation
  - Defines a set of attributes and methods that characterize any object of the class
  - Attributes and methods are members that are accessed via dot notation
- **Object or Instance :**
  - An individual object of a certain class which has own identity
  - A unique instance of a data structure that's defined by its class
  - State represents current values of attributes
  - State differs per object, behavior is shared by all instances
- **Lowering of semantic gap :**
  - Classes and objects can be discovered from domain of application
  - Application then uses concepts also present in real world

Object Oriented Concepts

- **Typical Object Oriented language concepts are the following**
- **Identity :**
  - Each object is distinct from each other object, and can be referred to
  - Two objects are distinct even if they have the same data
- **Encapsulation :**
  - Details can be hidden in classes
  - Programmers do not need to know all the internals of a class to use a class
  - Allows change in the internal implementation of class without changes in caller
- **Inheritance :**
  - Subclass inherits attributes and methods from a superclass
  - New attributes and methods may be added
  - Mechanism where features in a hierarchy inherit from superclasses to subclasses
- **Polymorphism**
  - Methods in a hierarchy can have same name but a different implementation
  - Version of method called is determined by actual object present

© copyright : spiraltrain@gmail.com

Example Class and Objects

	Car Class	Object Car A	Object Car B
Instance Variables	Plate Number	ABC 111	XYX 123
	Color	Blue	Red
	Manufacturer	Mitsubishi	Toyota
	Current Speed	50 km/h	100 km/h
Instance Methods	Accelerate Method		
	Turn Method		
	Brake Method		

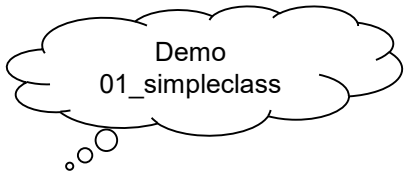
Creating Classes

- `class` **keyword creates a new class definition :**
  - Name of the class immediately follows the keyword `class` followed by a colon :

```
class ClassName:
    'Optional class documentation string'
    class_code
```

- **The class may have a documentation string :**
  - Can be access via `ClassName.__doc__`
- `class_code` **may consist of following language constructs :**
  - Class members, Data members, Method members
  - Properties, Constructor, Destructor

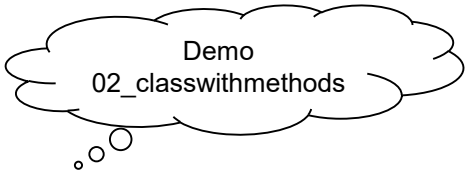
```
class Person:
    """Simple Person class"""
    pass  # An empty block
p1 = Person()
print(Person.__doc__)
print(p1)
```



Class with Methods

- **Instance methods in class are defined with `def` followed by colon :**
  - Instance methods work on individual object and have `self` as parameter
  - No need to include this argument when you call methods since Python adds it
- `self` **must be explicitly listed as first argument to method :**
  - Instance variables are referred to with `self.<instance_variable>`
- `self` **is a reference to the instance :**
  - Reference to the container for state of object
  - In many OO languages, instance is hidden in method definitions
  - In Python, instance is visible and explicit in method definitions

```
class Person:
    def sayHi(self):
        print('Hello, how are you?')
    def set_name(self, name):
        self.name = name
    def get_name(self):
        return self.name
    def greet(self):
        print("Hello, world! I'm %s." % self.name)
```



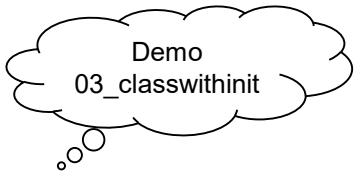
Object Initializers

- **Object initializer is special method called when object is created :**
  - In Python special method `__init__(self, …)` has role of object initializer
- **Create instances of a class :**
  - Assign variable to name of class passing parameters required by `__init__` :

```
class Computer:
    def __init__(self, name, price):
        self.name = name
        self.price = price
    def calculate(self):
        print('%s computer is calculating' %(self.name))
```

- **Initializer is typically used to create data member for objects :**
  - Can check whether parameters passed to `__init__` have valid values
- **Create object of `Computer` class and call member function :**

```
c = Computer('Apple', 1000)
c.calculate()
```



Accessing Attributes

- **In Python can add, remove, or modify attributes at any time :**

```
emp1.age = 7  # Add an 'age' attribute
emp1.age = 8  # Modify 'age' attribute
del emp1.age  # Delete 'age' attribute
```

- **There are alternative functions to access attributes :**
  - `getattr(obj, name[, default])` : Access the attribute of object
  - `hasattr(obj,name)` : Check if attribute exists or not
  - `setattr(obj,name,value)` : Set or create attribute
  - `delattr(obj, name)` : Delete an attribute
- **Example code :**

```
hasattr(emp1, 'age')    # Returns true if 'age' attribute exists
getattr(emp1, 'age')    # Returns value of 'age' attribute
setattr(emp1, 'age', 8) # Set attribute 'age' at 8
delattr(empl, 'age')    # Delete attribute 'age'
```

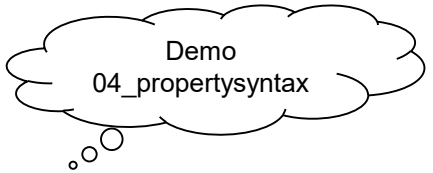
Property Syntax

- **Attributes of class can be accessed using get and set functions :**
  - Allow set function to have side effect such as a value check

```
class SquareGetterSetter:
    def get_length(self):           # Java-style
        return self._length
    def set_length(self, length):   # Java-style
        self._length = length
```

- **However Python also supports property syntax :**
  - Setters and getters are hidden by propery syntax and this is more pythonic way
  - May be added after class is used by clients without breaking them

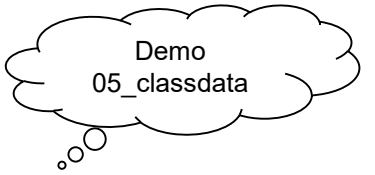
```
class SquarePropertySyntax:
    def get_length(self):
        return self._length
    def set_length(self, length):
        self._length = length
    length = property(get_length, set_length)
```



Class Data

- **Not all data are instance data belonging to objects :**
  - Sometimes data values should be shared among all instances of a class
- **Class data is called static data in other languages :**
  - An example is a counter that keeps track of how many objects are present
- **Class data is declared under `class` keyword outside constructor :**
  - Accessible by prefixing them with name of the class from inside or outside the class
- **Accessing class data like instance data is undesirable :**
  - Can declare a member function that returns class data

```
class C:
    count = 0   # number of times C.__init__ called
    def __init__(self):
        C.count = C.count + 1
    def getcount(self):
        return C.count  # or return self.count
c1 = C()
print('Current count:', C.count)
c2 = C()
print('Current count:', C.count)
```

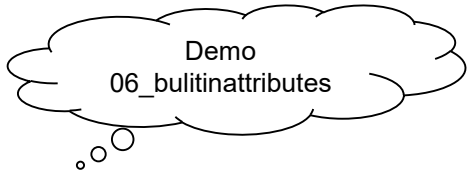


Built-in Class Attributes

```
p = Point()
print("Point.__dict__ :", Point.__dict__)
print("Point.__doc__ :", Point.__doc__)
print("Point.__name__ :", Point.__name__)
print("Point.__module__ :", Point.__module__)
print("Point.__bases__ :", Point.__bases__)
```

- **Result :**

```
Point.__dict__ : {'__module__': '__main__', 'count': 1, '__init__': <function
   Point.__init__ at 0x0000026112230378>, 'show_point': <function
   Point.show_point at 0x0000026112230268>, '__dict__': <attribute '__dict__'
   of 'Point' objects>, '__weakref__': <attribute '__weakref__' of 'Point'
   objects>, '__doc__': None}
Point.__doc__ : None
Point.__name__ : Point
Point.__module__ : __main__
Point.__bases__ : (<class 'object'>,
```



Destroying Objects

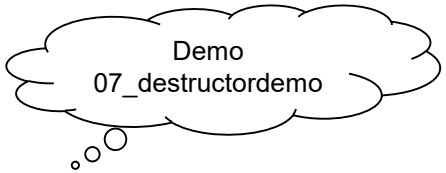
- **Python deletes unneeded objects automatically to free space :**
  - This process of periodically reclaims of memory is termed Garbage Collection
- **Python's garbage collector runs during program execution :**
  - Triggered when an object's reference count reaches zero
- **Reference count changes as nr of aliases pointing to it change :**
  - Increase when it's assigned a new name
  - Increase when placed in a container (list, tuple, or dictionary)
  - Decrease when deleted with `del`
  - Decrease when reference is reassigned or its reference goes out of scope
- **Reference count reaches zero :**
  - Python collects it automatically
- `__del__()` **destructor :**
  - May print the class name of an instance that is about to be destroyed
  - Might be used to clean up any non-memory resources used by an instance

Example Destructor

```
class Point:
   def __init__( self, x=0, y=0):
      self.x = x
      self.y = y
   def __del__(self):
      class_name = self.__class__.__name__
      print (class_name, "destroyed")
pt1 = Point()
pt2 = pt1
pt3 = pt1
print (id(pt1), id(pt2), id(pt3)) # prints id's of objects
del pt1
del pt2
del pt3
```

- **Result :**

```
3083401324 3083401324 3083401324
Point destroyed
```



Static Methods

- **Static methods do not receive `self` as first parameter :**
  - May access data associated with the class instead of instance
- **Method can be made `static` :**
  - Annotate method with `@staticmethod` decoration

```
class Account:
    interestRate = 10

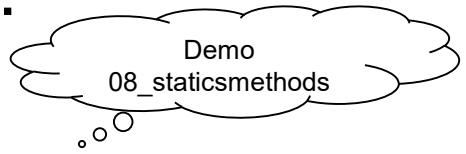
    def __init__(self, name):
        self.name = name

    def get_name(self):
        return self.name

    @staticmethod
    def get_interest_rate():
        return Account.interestRate
```

- `static` **method typically is called through class :**

```
print(Account.get_interest_rate())
Account.interestRate = 11
```

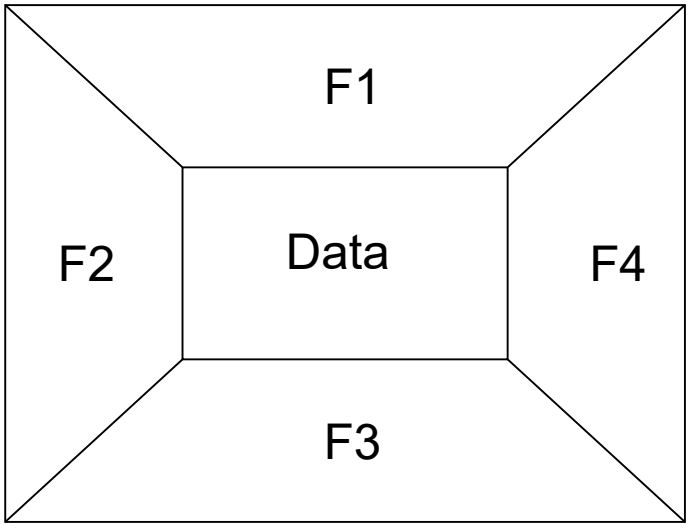


Naming Conventions

- **Names starting with underscore `_*` :**
  - Treated as `private` and not imported by from module `import *`.
- **Names starting and ending with double underscore `__*__` :**
  - Name of special method or attribute
  - Defined by interpreter and its implementation (including standard library)
- **Examples of special methods and attributes :**
  - `__init__`, `__del__`
  - `__doc__`, `__module__`, `__name__`
- **Names starting with double underscore in class `__*` :**
  - Class-private when used within the context of a class definition
  - Mangled to avoid name clashes between private attributes of base and derived classes
- **Stylistic (PEP8) conventions :**
  - Use underscores to separate multi-word names: `name_of_some_variable`
  - Capitalize classnames: `Some_Class`

Encapsulation

- **Well known object oriented concept**
- **Internal class data hidden by functions**
- **Result is implementation hiding and modularity**



Data Hiding

- **Many object oriented languages support data hiding :**
  - Attributes and methods may have modifiers like public, private and protected
- **Python does not support real data hiding as other languages :**
  - Attributes and methods can however be made more difficult to access
- **Attributes made private with double underscore prefix in name :**

```
self.__secretCount = 0
```

- **Attributes will not be directly visible to outsiders :**
  - Access of such attributes from outside should include class name

```
print(s._Secretive__secretCount)
```

- **Methods can also be protected with double underscore as prefix :**

```
def __inaccessible(self):
    print("Bet you can't see me...")
```

- **Methods can accessed from the outside in the following way :**

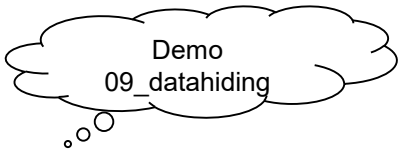
```
s._Secretive__inaccessible()
```

Example Data Hiding

```
class Secretive:
    def __init__(self):
        self.__secretCount = 0
    def count(self):
        self.__secretCount += 1
        print(self.__secretCount)
    def __inaccessible(self):
        print("Bet you can't see me...")
    def accessible(self):
        print("The secret message is: ", end='')
        self.__inaccessible()
s = Secretive()
s.count()
s.count()
print (s.__secretCount)
```

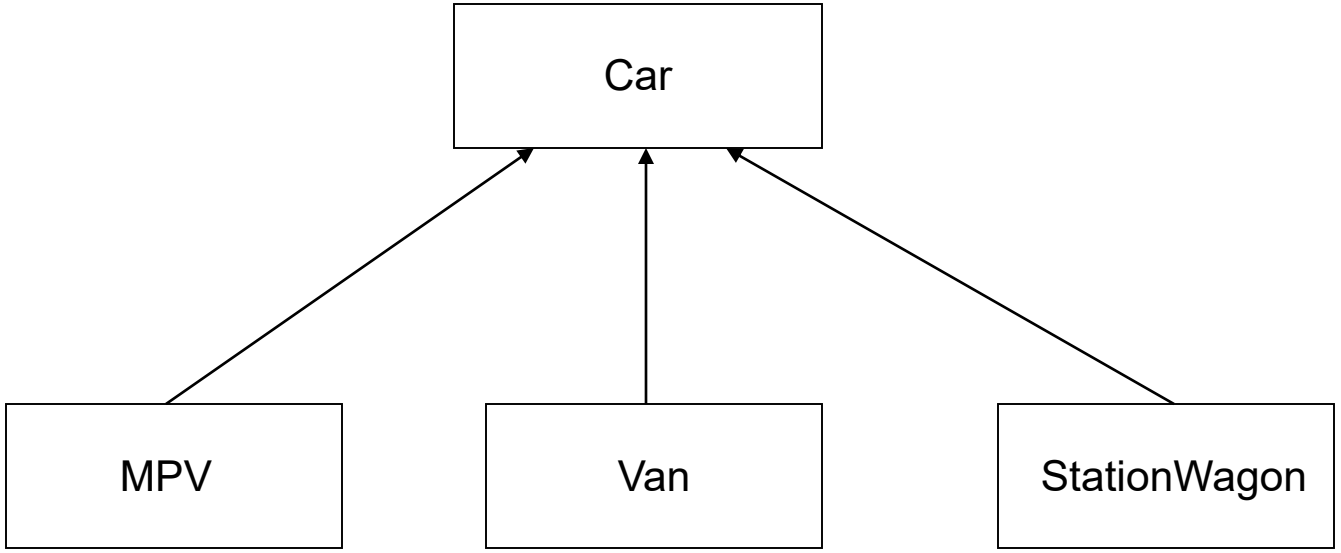
- **Result :**

```
1
2
Traceback (most recent call last):
    AttributeError: 'Secretive' object has no attribute '__secretCount'
```



Inheritance

- Well known object oriented concept
- Subclass inherits attributes and methods from a superclass
- New attributes and methods may be added
- Subclass may provide new version of superclass methods
- Benefits of inheritance are code reuse and less bugs
- Forms hierarchical relationships between classes



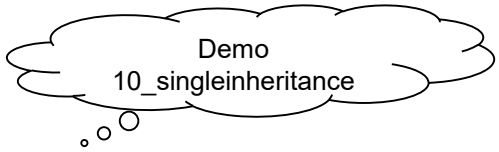
Inheritance in Python

- **Class can be derived from a preexisting parent class :**
  - List parent class in parentheses after the class name
  - Python supports multiple inheritance
  - List of base classes to inherit from may be present
- **Child class inherits the attributes of its parent class :**
  - Attributes can be used as if they were defined in the child class
  - Child class can also override data members and methods from the parent
- **Derived classes are declared much like their parent class :**

```
class SubClassName (ParentClass1[, ParentClass2, ...]):
   'Optional class documentation string'
   class_code
```

- **Base class constructor call from derived class constructor :**
  - Prefix `__init__` with keyword `super` or with name of base class

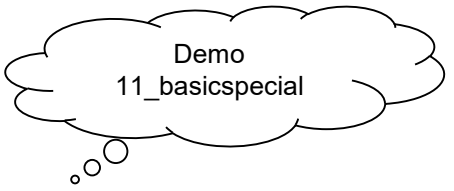
```
super().__init__(name)
Car.__init__(self ,name)
```



Deriving Classes

- **Class Special that inherits from super class Basic :**
  - Constructor base class must be called explicitly with prefix `super` or base class name

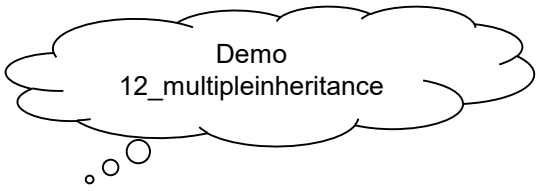
```
class Basic:
    def __init__(self, name): self.name = name
    def show(self): print 'Basic name: %s' % self.name
class Special(Basic):
    def __init__(self, name, edible):
        # super().__init__(name)
        Basic.__init__(self, name)
        self.upper = name.upper()
        self.edible = edible
    def show(self):
        # super().show()
        Basic.show(self)
        print 'Special upper name: %s.' % self.upper,
        if self.edible:
            print "It's edible."
        else:
            print "It's not edible."
    def edible(self):
        return self.edible
```



Multiple Inheritance

- **Python supports multiple inheritance :**
  - Class can be derived from multiple parent classes
- **Class inherits attributes and methods from multiple parent classes :**
  - Constructors of base classes must be called explicitly
- **Ambiguities if same attributes occur in different base classes :**
  - Therefore multiple inheritance is banned from many languages like Java and C#

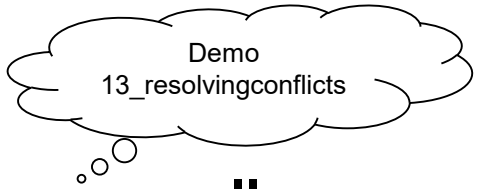
```
class Person:
    def __init__(self, name, age): self.name = name; self.age = age
    def show_name(self): print(self.name)
    def show_age(self): print(self.age)
class Student:
    def __init__(self, student_id) : self.student_id = student_id
    def get_id(self): return self.student_id
class Resident(Person, Student): # extends both Person and Student class
    def __init__(self, name, age, id):
        Person.__init__(self, name, age)
        Student.__init__(self, id)
res = Resident('Robert', 30, '102')
res.show_name()
print(res.get_id())
```



Resolving Conflicts Multiple Inheritance

- **If same attribute present in multiple base classes :**
  - Attribute value in derived class depends on order of base class constructor calls
  - Last constructor call that sets attribute wins, it overrides previous constructor call

```
class A:
    def __init__(self):
        self.name = 'Albert'
    def getName(self):  return self.name
class B:
    def __init__(self):
        self.name = 'Enrico'
    def getName(self): return self.name
class C(A, B):
    def __init__(self):
        A.__init__(self)
        B.__init__(self)       # B wins thus Enrico
    def getName(self):
        return self.name
c1 = C()
print(c1.getName())
```



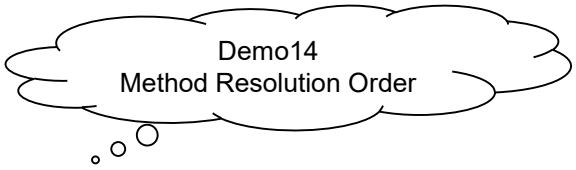
- **Hierarchy becomes depended on order of `__init__()` calls :**
  - Result would be the same for `C(B,A)`

Method Resolution Order

- **MRO specifies lookup path for attributes and methods :**
  - Works in a depth first left to right way
- **MRO computes proper monotonic linear search order :**
  - In case indirect base classes can be reached along different paths
  - Gives error in case of ambiguous results
- **Call `print(C.__mro__)` to see the MRO traceroute**

```
class A:
    def foo(self): print("A")
class B:
    def foo(self): print("B")
class C (A, B):
    pass
class D (B, A):
    pass
# TypeError: no consistent method resolution order found
class E (C, D):
    pass
```

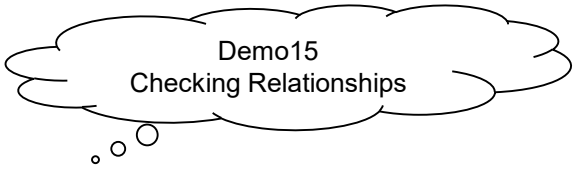
```
    class E(C, D):
TypeError: Cannot create a consistent method resolution
order (MRO) for bases B, A
```



Checking Relationships

- **Functions to check relationships of classes and instances :**
  - `issubclass()` or `isinstance()`
- `issubclass(sub, sup)` **is boolean function :**
  - Returns true if the given subclass sub is a subclass of the superclass sup
  - Check if a class type is the subclass of a different class
- `isinstance(obj, Class)` **is boolean function :**
  - Returns true if obj is instance of class Class or is instance of a subclass of Class
  - If an object is an instance of a certain class or any of its subclass

```
class Polygon:
    pass
class Triangle(Polygon):
    pass
class Circle(Polygon):
    pass
t = Triangle()
c = Circle()
print("Triangle is subclass of Polygon %s" %(issubclass(Triangle, Polygon))) #True
print("c is instance of Circle %s" %(isinstance(c, Circle)))                 #True
```



Overriding Methods

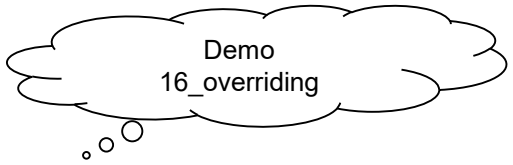
- **Always possible to override parent class methods :**
  - Reason could be that special or different functionality in your subclass is needed

```
class Parent:        # define parent class
   def myMethod(self):
      print 'Calling parent method'
class Child(Parent): # define child class
   def myMethod(self):
      print 'Calling child method'

c = Child()          # instance of child
c.myMethod()         # child calls overridden method
```

- **Result :**

```
Calling child method
```



Override Base Methods

- **Generic functionality that you can override in your own classes :**
- `__init__(self [,args...]):`
  - Initializer with any optional arguments which handles object initialization

    ```
    obj = className(args)
    ```
- `__new__(cls):`
  - Constructor handling object creation has `cls` as parameter since `self` not yet there

    ```
    obj = className(args)
    ```
- `__del__(self) :`
  - Destructor, deletes an object

    ```
    dell obj
    ```
- `__repr__(self):`
  - Evaluatable string representation

    ```
    repr(obj)
    ```
- `__str__(self) :`
  - Printable string representation

    ```
    str(obj)
    ```

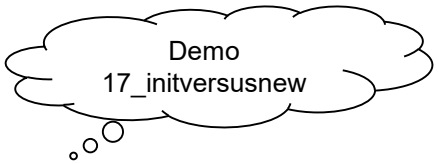
__init__ versus __new__

- **Both `__new__` and `__init__` can be overridden :**
  - `__new__` is the constructor and is solely for creating the object
  - `__init__` is the initializer is for initializing the object
- **`__new__` accepts `cls` as parameter :**
  - `__new__` is called each time when calling the class name
  - Should return call to `super.__new__(cls)` otherwise `__init__` not called
- **`__init__` accepts `self` as parameter :**
  - Called every time instance of the class is returned by __new__
  - Passing the returned instance to `__init__` as the `self` parameter

```
class A:
    def __new__(cls):
        print("A.__new__ called")
        return super().__new__(cls)
    def __init__(self):
        print("A.__init__ called")

a1 = A()
a2 = A()
```

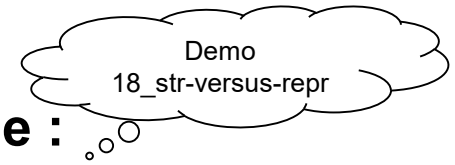
```
A.__new__ called
A.__init__ called
A.__new__ called
A.__init__ called
```



__str__ versus __repr__

- **Both `__repr__` and `__str__` are used to represent objects :**
  - But in different ways, no difference for numbers, difference for strings
- **`__repr__` built-in function :**
  - Computes the "official" string reputation of an object
- **`__str__` built-in function :**
  - Computes the "informal" string representations of an object
- **Best way to understand difference is to see them in action :**

```
x = 4
print(repr(x))
print(type(repr(x)))
print(str(x))
print(type(str(x)))
y = 'stringy'
print(repr(y))          # note the quotes 'string'
print(type(repr(y)))
print(str(y))           # note no quotes string
print(type(str(y)))
```



- **Default implementation of `__repr__` for a `str` value :**
  - Can be called as argument to `eval` and result in valid string

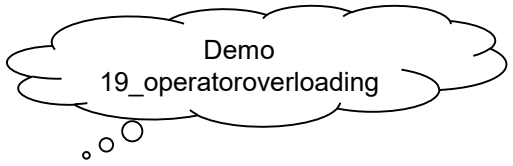
Operator Overloading

- **Suppose `Vector` class represents two-dimensional vectors :**
  - Plus operator will not work to add them automatically
- **Define `__add__` method to perform vector addition :**
  - Plus operator would behave as per expectation

```
class Vector:
   def __init__(self, a, b):
      self.a = a
      self.b = b
   def __str__(self):
      return 'Vector (%d, %d)' % (self.a, self.b)
   def __add__(self,other):
      return Vector(self.a + other.a, self.b + other.b)

v1 = Vector(2,10)
v2 = Vector(5,-2)
print v1 + v2
```

- **Result** : `Vector(7,8)`



Polymorphism

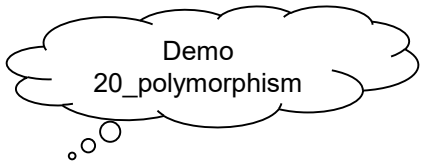
- Well known object oriented concept
- The ability to assume different forms
- Set of related objects behave in similar but independent ways
- Same method present in different classes of hierarchy
- Generic method call results in specific runtime action

```
cir = Circle(1, 2)
rec = Rectangle(3, 4, 2, 2)
list = [cir, rec]

for item in list :
    item.draw();
```

Polymorphism in Python

```
class A(object):
    def show(self, msg):
        print 'class A -- msg: "%s"' % (msg, )
class B(object):
    def show(self, msg):
        print 'class B -- msg: "%s"' % (msg, )
class C(object):
    def show(self, msg):
        print 'class C -- msg: "%s"' % (msg, )
def test():
    objs = [A(), B(), C(), A(), ]
    for idx, obj in enumerate(objs):
        msg = 'message # %d' % (idx + 1, )
        obj.show(msg)
if __name__ == '__main__':
    test()
```



Abstract Classes

- **Abstract classes contain one or more abstract methods :**
  - An abstract method is a method that is declared, but contains no implementation
- **Abstract classes may not be instantiated :**
  - They require subclasses to provide implementations for the abstract methods
- **Python does not provide abstract classes :**
  - However `abc` module provides infrastructure for Abstract Base Classes (ABCs)
- **Abstract Base Classes (ABC) :**
  - Specify interfaces object promises to implement, permit inspection before invocation
- **Abstract methods are decorated with `@abstractmethod` :**
  - They have no implementation
- **Derived classes must provide the implementation :**
  - Otherwise the derived class cannot be instantiated
- **Python library has useful ABCs that we can easily reuse :**
  - Can get a list of useful built in ABCs in the `collections.abc`

Abstract Method

- **ABC module provides infrastructure for abstract base classes :**

```
from abc import ABC, abstractmethod
class AbstractClass(ABC):
    def __init__(self, value):
        self.value = value
@abstractmethod
    def do_something(self):
        pass
class C(AbstractClass):
    # pass
    def do_something(self):
        return self.value + 42

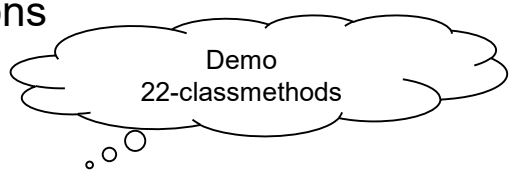
c = C(22)
print(c)
print(c.do_something())
```

- **Object of class C must implement abstract method :**
  - Otherwise class cannot be instantiated
- **Class consisting of only abstract methods :**
  - Considered an interface that derived classes must implement

Demo
21_abstractmethods

Class Methods

- **Class method receive class as implicit first argument :**
  - Just like an instance method receives the instance
- **Class methods are decorated with `@classmethod` :**
  - Have access to the state of the class as it takes class parameter
  - Can modify class state that would apply across all the instances of the class
- **Class methods versus static methods :**
  - Static methods can't access or modify class state but class methods can
  - Class methods are typically used to create factory methods that return class objects
  - Static methods are typically used to create utility functions




```
class Person:
     def __init__(self, name, age):
        self.name = name; self.age = age
    @classmethod # class method to create a Person object by birth year.
    def fromBirthYear(cls, name, year):
        return cls(name, date.today().year - year)
    @staticmethod # static method to check if a Person is adult or not.
    def isAdult(age):   return age > 18
```

Summary : Advanced Classes

- **Objects are instances of classes :**
  - Each object has its own state and identity, instance methods have `self` parameter
- **To get and set attributes Python supports property syntax :**
  - Setters and getters are hidden by propery syntax and this is more pythonic way
- **Python deletes unneeded objects automatically to free space :**
  - This process of periodically reclaims of memory is termed Garbage Collection
- **Encapsulation is hiding internal data through methods :**
  - `private` instance variables have double underscores `__var`
  - Can be accessed through a scrambled name
- **Static methods and properties bound to class not to object :**
  - Static data only exists once per class rather than per instance
- **Class can be derived from a preexisting parent class :**
  - List parent class in parentheses after the class name
- **Class can be derived from multiple parent classes :**
  - Check relationships with `issubclass()` or `isinstance()`

